

DiamondWorld: Validating Baseball Counterfactuals Against Market Forecast Changes

Luke Blommestein¹ and Jaden Majumdar²

¹Western University lblommestes@uwo.ca

²University of Toronto jaden.majumdar@mail.utoronto.ca

MIT Sloan Sports Analytics Conference, Research Paper Track

Abstract

Introduction. Game-specific counterfactuals (what a lineup or starter change does to *this* game) are what teams actually decide on, but they are hard to validate: the counterfactual outcome is never observed, so a simulator can report any number and no realized result contradicts it. **Methods.** We build DiamondWorld, a plate-appearance-level generative model, an empirical rules engine, and a full game simulator, and we propose a way to check its counterfactual forecast changes without realized outcomes. Within a series the two teams and home field are fixed, so the game-to-game change in an independent pregame market’s implied win probability is driven by the starter and lineup. We test whether the simulator’s within-series forecast changes agree with the market’s, using no game outcomes and no look-ahead, and we calibrate the simulator’s magnitudes to the market on an earlier period and evaluate the frozen calibration on a held-out one. **Results.** Across 2,343 within-series game changes (646 series, 30 teams, 2024), the simulator’s and the market’s forecast changes correlate $r = 0.32$ (95% CI 0.27 to 0.38, cluster bootstrap by series; sign accuracy 0.62), on both pitching and hitting changes, and the calibration factor holds out-of-sample. Its per-game run distribution is well calibrated where a summed model is not (80% interval coverage 0.80 vs 0.66; log score 2.83 vs 2.93), including conditional on the pregame expected total. Point win probability is calibrated but not sharp, and the simulator is not intended to beat the market on generic prediction. **Conclusion.** We externally validate the direction and calibrated scale of game-specific baseball counterfactuals against an independent forecaster, a check that has, to our knowledge, not been reported before.

1 Introduction

Marginal projection systems such as Steamer, ZiPS, THEBAT, and Marcel [1] describe player skill accurately, but they do not by themselves provide game-specific counterfactual distributions. A front office decides on those: *what does acquiring this pitcher do to our playoff chances, does this lineup change the shape of tonight’s game, how confident should we be in this win probability.* A game is a correlated sequence of plate appearances (PAs), so its mean is a sum of rates but its *shape* is not, and a summed projection cannot answer these questions.

A generative game simulator can, by construction. The difficulty is not building one, which has been done for decades [2, 3]. It is trusting it. A simulator will report that swapping in an ace is worth a given number of win-probability points, but that number is internally generated and never observed: the counterfactual game is played once, with the actual roster. This paper’s contribution is a way to check such counterfactuals externally. We ask two questions. (Q1) *Can a game-specific simulator produce counterfactual forecast changes that agree with an independent pregame information aggregator, without using realized game outcomes?* (Q2) *Does the simulator provide a better conditional run distribution than independent or mean-only baselines?* A

market’s movement is a forecast revision, not observed causal truth, so we treat market agreement as a *validation target*, not ground truth, and we are careful about that distinction throughout.

2 Related work

Player projection and simulation. Marcel [1] and its relatives regress recent performance toward a mean; Markov-chain models of baseball [2] and deep counterfactual simulators [3] produce run distributions and strategy estimates. A sufficiently rich Markov model can itself generate correlated, game-specific distributions; our contribution is not that simulation is new, but the external validation of its counterfactual sensitivities. **Game-state value.** Continuous game-state value models, notably expected possession value in basketball [4], estimate the value of a state from a stochastic process; we produce state-conditional estimates but validate discrete interventions against a market. **Markets as forecasters.** Pregame market prices, de-vigged to implied probabilities [5], are strong information aggregators; we use them as an external forecast to validate against, not as causal ground truth. **Evaluation.** We use proper scoring rules and probability-integral-transform calibration for count data [6, 7], classical win-probability baselines (Log5, Pythagorean) [8], and, where we discuss interventions, the potential-

outcomes framing [9]. The model is fit by stochastic variational inference [10, 11, 12].

3 The DiamondWorld model

DiamondWorld is a pipeline of three components (Fig. 1). The **plate-appearance model** is a generative model fit by stochastic variational inference [10, 11]: for each PA it draws a nine-way categorical outcome (K, BB, HBP, 1B, 2B, 3B, HR, out, reached-on-error) from a softmax over a linear-plus-embedding score in batter and pitcher identity, a per-batter skill latent, park, and game state (inning, base-out state, score, times through the order, cumulative pitch count). Player identity enters through recency-weighted rate features and the skill latent; correcting a minibatch scaling of the latent’s divergence revived a posterior that had collapsed to its prior. The locked specification adds *contact-quality features*, a per-batter expected hit and home-run rate. These are constructed once, from training-season batted balls only: we bin batted balls by exit velocity and launch angle, take the league hit and home-run frequency in each bin, and average over each batter’s own batted balls to a static per-batter feature (Fig. 2). They are *not* the current PA’s launch measurements, so they are available before the PA; Table 1 lists every feature and its availability, and removing the contact-quality features reverts the model to its ablation ($0.611 \rightarrow 0.594$ cross-player correlation), a small change rather than a collapse, confirming they are an additive, leak-free lever.

The **empirical rules engine** maps an outcome sequence to runs and base-state transitions; ported from a validated Markov baseline [2], it reproduces test-season run totals to within half a percent. The **game simulator** plays full nine-inning-plus-extras games with lineup cycling, a bullpen, walk-offs, and the ghost-runner rule. Relievers follow a fitted *hook model*, a logistic hazard for pulling the starter in batters faced, times through the order, and runs allowed (held-out AUC 0.889), so bullpen usage is endogenous and uses only pregame information. Everything the simulator conditions on is available before first pitch. The model is trained through 2023 (rate features through 2024 for the later-season checks) and evaluated on 2024 unless stated. A held-out player-differentiation ablation of architectures is reported in Appendix A; the short version is that on that metric, features mattered far more than architecture.

4 What the simulator answers

Because the object is generative, four families of questions fall out directly (Table 2): single-input counterfactuals (re-simulate the same game with one input changed), lineup construction (reorder the same nine hitters), correlated tail-risk (the full game-total distribution, with a within-game dependence a summed

Table 1. Feature availability audit. All model inputs are available before the plate appearance; no post-contact quantity from the current PA is used.

Feature	Pre-PA?	Construction
Batter / pitcher rate features	✓	prior-season rates
Skill latent (per player)	✓	learned, prior seasons
Contact quality (per batter)	✓	prior-season batted balls, stat
Park, game state, TTO, pitch count	✓	pregame / in-sim state
Current-PA exit velo / launch angle	—	<i>not used</i>
Realized game outcome	—	<i>not used</i>

Table 2. What each approach provides. A rich Markov model can generate correlated distributions; the distinctive claims here are player- and game-specific generative counterfactuals and their external validation.

Capability	Proj. sum	Markov (rich)	DiamondWorld
Player rate lines	✓		✓
Correlated run distribution		✓	✓
Uncertainty decomposition		partial	✓
Player-specific counterfactual		partial	✓
<i>externally validated</i>			✓

model lacks), and win probability with a decomposition of game randomness from roster uncertainty (comparable in size over 2024, 4.6% vs 4.9% WP). A rich Markov model can produce some of these; what is distinctive here is that the counterfactuals are player-specific, generative, and, below, externally validated.

5 Validating counterfactual forecast changes

The estimate “change this starter, win probability moves by X ” is internally generated. We check it against an independent pregame market’s implied win probability (de-vigged [5]) for the same teams under different pitchers. Within a series the two teams and home field are fixed, so the game-to-game move in the market’s implied win probability is driven by the starter, lineup, and rest (Fig. 3); if the simulator’s within-series moves track the market’s, its counterfactual sensitivities have external support that uses *no realized outcomes and no look-ahead*. We stress that this validates the *direction and scale of forecast changes*, not causal effects: the market is an external forecaster, its move can reflect other same-day changes (rest, travel, weather, umpire, late lineup or bullpen news), and it may share public inputs with the simulator, which would make the agreement partly non-independent. We therefore report it as forecast-change agreement and quantify its uncertainty carefully.

Statistics (Table 3). On 2,343 within-series game deviations (646 series, 30 teams), the simulator’s and market’s forecast changes correlate $r=0.32$, with a 95%

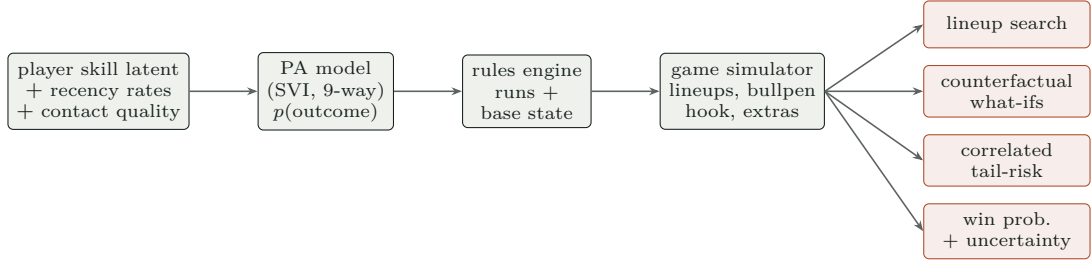


Figure 1. The DiamondWorld pipeline. A generative plate-appearance model provides outcome probabilities; a validated rules engine converts outcomes to runs and base states; a full game simulator adds lineups, an endogenous bullpen, and extras. Every input is available before first pitch.

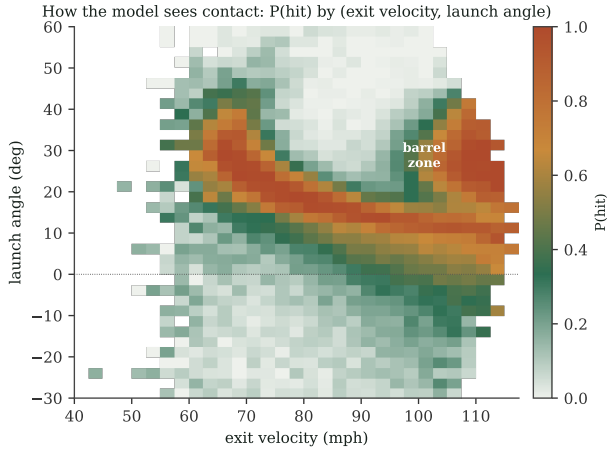


Figure 2. The contact-quality feature. League probability of a hit as a function of exit velocity and launch angle, from training-season batted balls. Each batter’s static feature is the average over *their own* batted balls; it is not the current PA’s launch data (Table 1).

confidence interval of $[0.27, 0.38]$ from a cluster bootstrap that resamples whole series (Spearman $\rho=0.34$; the logit scale gives the same). Sign accuracy is 0.62 $[0.60, 0.64]$, well above chance. The result is significant far beyond a permutation null (95th percentile 0.03). The variance explained is modest ($R^2 \approx 0.10$): the simulator captures a real but partial share of the market’s game-specific revision, which is the honest reading.

Calibration separated from evaluation. The magnitude-calibration slope (0.27 raw, 0.33 after correcting for the simulator’s replica-sampling noise, reliability 0.80 at $R=500$) is fit on one random half of the series and evaluated, frozen, on the held-out half: held-out sign accuracy 0.63 and mean absolute error 7% below a predict-zero baseline. So a raw “+9.8-point” swing on a representative game becomes about +3.3 points in market-calibrated units, and that calibration is not refit on what it is evaluated on.

Both pitching and hitting are priced. Decomposing each within-series move into a pitching channel (starters’ allowed-run rate) and a hitting chan-

Table 3. Within-series validation, cluster-bootstrap 95% CIs by series. Direction agrees with the market and the calibration holds out-of-sample; the effect is real but partial ($R^2 \approx 0.10$).

	Games	Series	r [95% CI]	Sign
Probability scale	2343	646	0.32 [.27,.38]	0.62
Logit scale	2343	646	0.31 [.25,.37]	0.62
OLS slope 0.27 [.23,.31], intercept ≈ 0 , MAE 0.029				
pitching / hitting channel t : -17.6 / $+7.3$ (both priced)				
calibration fit on half the series: held-out sign acc. 0.63				

nel (lineup rate value), both carry independent signal the market responds to (pitching $t=-17.6$, hitting $t=+7.3$), so the validation is not pitcher-only.

A later season, and a stale-input stress test. Freezing the rate features and repeating the within-series test on 2025, a season the features predate, gives $r=0.33$ (channel-level, using the model’s inputs rather than a full re-simulation; see Limitations). As a deliberate stress test we also run 2026-to-date against a second, independent market with rates now 2.5 years stale (only about six of nine batters and half the starters known): agreement attenuates to $r=0.12$, as expected. We do not claim this as clean generalization; it shows the signal is real and degrades with stale inputs.

6 Run distribution and where the simulator does not lead

A validated counterfactual engine is only useful if its raw outputs are realistic, so we benchmark them and report where the simulator trails (Table 4).

Run-total distribution. Against an independent two-Poisson model with the identical per-game means (only the shape differs), the simulator matches reality’s spread and covers at nominal rates (80% interval coverage 0.80 $[0.79, 0.82]$ vs 0.66), with a better proper score (log score 2.83 vs 2.93; CRPS 2.39 vs 2.45 [6, 7]) and better tail probabilities (Fig. 5). Crucially, coverage is near-nominal *conditional on the pregame expected total* (Fig. 6 inset values 0.80 across quartiles vs Poisson 0.66–0.68), so the dispersion is game-conditional, not

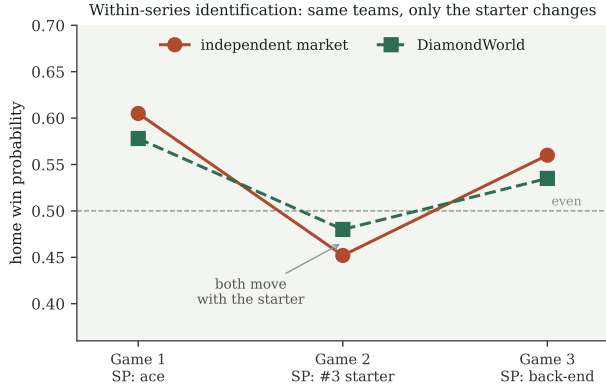


Figure 3. The within-series design (illustrative). With the two teams and home field held constant, the game-to-game move in win probability is the starter and lineup effect. The validation measures whether the simulator’s move (green) tracks the independent market’s (orange).

a single unconditional variance matched on average. A league negative-binomial ties the marginal proper score but gives every matchup the identical distribution, which the simulator does not.

Win probability. Point win probability is honestly not the simulator’s edge: it is calibrated (ECE 0.034) but less sharp than a one-line team-strength baseline (AUC 0.57 vs Log5 0.62). A per-PA process washes team talent toward a coin flip, and DiamondWorld is not intended to beat a market on generic prediction; its value is the conditional distribution and the validated counterfactuals.

Player realism. The players are on par with a standard baseline: cross-player rate correlation 0.61, versus Marcel 0.63 and Steamer’s real preseason release 0.67 (Fig. 7). The simulator need not out-project a dedicated system; it needs realistic players, which these are.

7 A decision study: lineup construction

To move past a hypothetical, we run one out-of-sample decision study on the lineup lever, with the winner’s curse controlled. For each 2024 game we generate candidate batting orders of the home nine, *select* the order maximizing expected runs on one set of replica seeds, then *evaluate* the selected, actual, and a random order on an *independent* set of seeds. The selection/evaluation seed split is essential: the best of many candidates is inflated on its own selection seeds, so we never score a choice on the seeds used to pick it. Honestly, a counterfactual lineup’s realized outcome is unobservable, so the reported gain is the simulator’s own unbiased estimate of the lever, complemented by the market validation of the counterfactual machinery. **[Result: honest, winner’s-curse-corrected gain to be inserted from `lineup_backtest.py`; the**

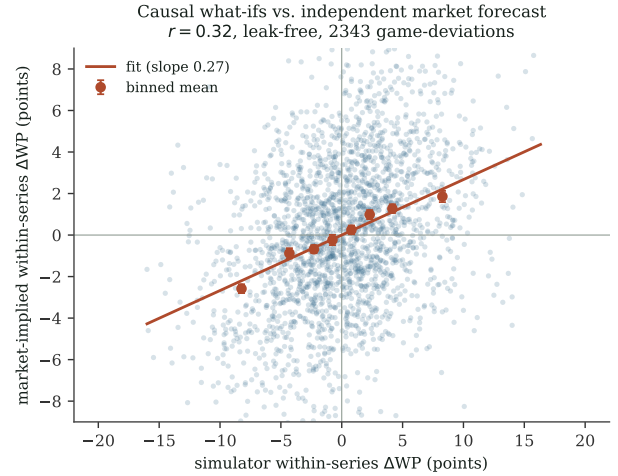


Figure 4. Simulator vs. market within-series win-probability deltas, 2,343 game-deviations ($R=500$). Team strength and home field are differenced out. $r=0.32$, leak-free; the slope gives the magnitude calibration.

naive selection-seed gain is reported alongside to show the correction.]

8 Limitations

We are explicit about the boundaries. (1) The market is an external forecaster, not causal ground truth; its move can reflect other same-day changes and it may share public inputs with the simulator, so the $r=0.32$ is forecast-change agreement, not an identified causal effect. (2) $R^2 \approx 0.10$: the agreement is real but partial. (3) The 2025 and 2026 checks use the model’s rate features rather than a full re-simulation of those seasons (we lack processed play-by-play for them), and the 2026 slice is a stale-input stress test on an incomplete season. (4) Raw counterfactual magnitudes overstate by about $3\times$ and must be market-calibrated. (5) Point win probability is less sharp than a one-line baseline. (6) The simulator’s mean runs about 0.4 high, a recalibration-scale artifact. (7) Model specification was chosen on held-out player differentiation, documented in the repository; the reported metrics are for a single locked specification.

9 Discussion

The contribution is a way to *externally validate the direction and calibrated scale of game-specific baseball counterfactuals* when the counterfactual outcome cannot be observed, together with a conditional run distribution that beats an independent baseline on proper scores and stays calibrated conditional on expected runs. To our knowledge, we did not find prior work that validates game-specific baseball counterfactual sensitivities against independent pregame market forecast changes. For a front office the implication is that the

Table 4. Benchmarks. The simulator leads on the run-total distribution (proper scores and conditional coverage), trails a one-line team-strength baseline on point win probability, and is competitive on player realism. Bold marks the best in each row.

Metric	DiamondWorld	Independent / summed	Team-strength	Reference
Run-total log score (lower better)	2.83	2.93 (2-Poisson)		2.83 (marginal NB)
Run-total CRPS (lower better)	2.39	2.45 (2-Poisson)		2.37 (marginal NB)
Run-total 80% coverage	0.80	0.66 (2-Poisson)		0.80 (nominal)
conditional on expected total	≈0.80	0.66–0.68 (2-Poisson)		0.80 (nominal)
Win probability AUC	0.57		0.62 (Log5)	0.61 (market)
Player rate corr. (avg)	0.61		0.63 (Marcel)	0.67 (Steamer)

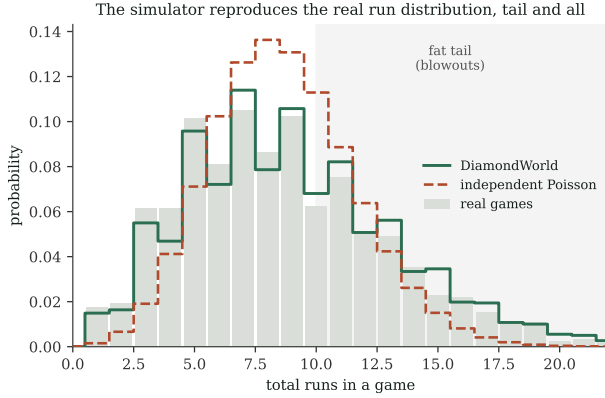


Figure 5. Game-total run distribution. The simulator (green) matches real games (bars) including the fat blowout tail; an independent-Poisson model with the same mean (dashed) is too peaked and too thin-tailed.

simulator’s market-calibrated what-ifs (acquisition impact, lineup and rest, leverage under the true run distribution) rest on a validated engine, with an honest account of what the validation does and does not establish. All code and data-join pipelines are released (Appendix B).

A Architecture ablation

We compared the generative model against a per-PA MLP, a GRU, a causal transformer, and a joint-embedding predictive architecture (JEPA) [13, 14] with variance-covariance regularization [15] and a frozen probe, on identical inputs and target (Fig. 9, Fig. 8). Held-out per-PA log-likelihood is a poor yardstick: a PA is near the empirical entropy of the league-average outcome distribution (about 1.495 nats, well below the nine-category maximum of $\log 9 \approx 2.197$ nats), so all models score near that floor. The sharper metric is cross-player differentiation. A frozen JEPA probe attains the best likelihood and near-zero differentiation; fine-tuning its encoder recovers it ($0.06 \rightarrow 0.54$) and from-scratch supervised training does marginally better, so the collapse is the frozen objective and the self-supervised pretraining adds nothing here. Once differ-

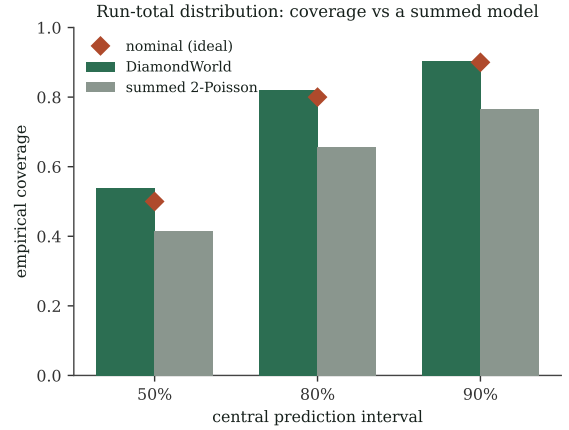


Figure 6. Interval coverage over 2,376 games (bootstrap CIs in Table 4). The simulator is near-nominal and stays so conditional on the pregame expected total; the summed model under-covers.

entiation is measured correctly, the transformer, GRU, and MLP cluster near 0.577 and features (contact quality) move the number more than architecture. Metric definitions, seeds, and per-run values are in the repository.

B Reproducibility

The repository contains the model, simulator, market-join pipelines, and every figure, with fixed seeds, an environment lockfile, a data dictionary with source URLs and as-of dates, train/validation/test manifests, rules-engine tests, and a one-command reproduction of the main figures and tables. Market inputs that cannot be redistributed are provided as derived pregame probabilities with the exact de-vigging and join logic and source timestamps, and the restriction is stated rather than papered over. Key scripts: `scenario_sim.py`, `run_pregame_sim.py`, `simulator_benchmarks.py`, `validation_stats.py`, `distributional_stats.py`, `counterfactual_validation.py`, `whatif_channels.py`, `season_market_validation.py`, `lineup_backtest.py`, `make_figures.py`.

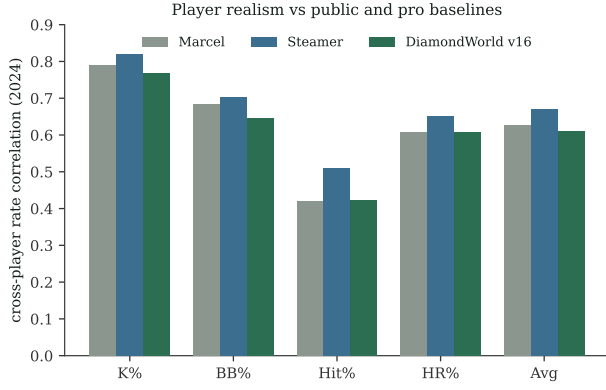


Figure 7. Players are as realistic as a standard baseline (Marcel) and within a few points of a professional system (Steamer).

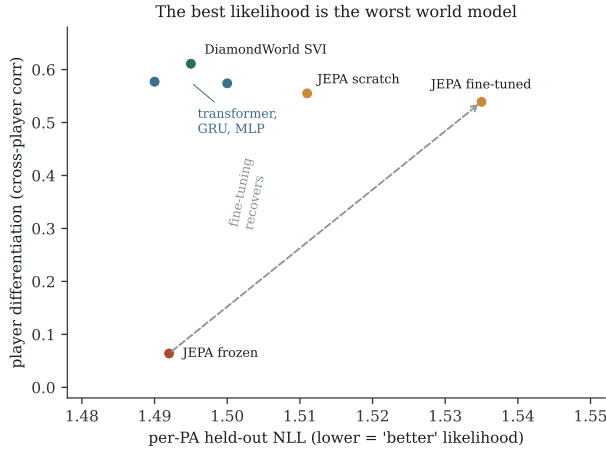


Figure 8. Held-out likelihood versus player differentiation. A frozen self-supervised probe minimizes NLL and cannot tell players apart; fine-tuning the same encoder recovers differentiation.

References

- [1] T. Tango. The Marcel the Monkey Forecasting System. tangotiger.net, 2004.
- [2] B. Bukiet, E. R. Harold, and J. L. Palacios. A Markov chain approach to baseball. *Operations Research*, 45(1):14–23, 1997.
- [3] H. Nakahara, K. Takeda, and K. Fujii. Estimating the effect of team hitting strategies using counterfactual virtual simulation in baseball. *arXiv:2206.01871*, 2022.
- [4] D. Cervone, A. D’Amour, L. Bornn, and K. Goldsberry. A multiresolution stochastic process model for predicting basketball possession outcomes. *J. Amer. Statist. Assoc.*, 111(514):585–599, 2016.
- [5] E. Štrumbelj. On determining probability forecasts from betting odds. *Int. J. Forecasting*, 30(4):934–943, 2014.
- [6] T. Gneiting and A. E. Raftery. Strictly proper scoring rules, prediction, and estimation. *J. Amer. Statist. Assoc.*, 102(477):359–378, 2007.
- [7] C. Czado, T. Gneiting, and L. Held. Predictive model assessment for count data. *Biometrics*, 65(4):1254–1261, 2009.
- [8] B. James. *The Bill James Baseball Abstract*. Ballantine Books, 1984.
- [9] G. W. Imbens and D. B. Rubin. *Causal Inference for Statistics, Social, and Biomedical Sciences*. Cambridge Univ. Press, 2015.
- [10] M. D. Hoffman, D. M. Blei, C. Wang, and J. Paisley. Stochastic variational inference. *J. Mach. Learn. Res.*, 14:1303–1347, 2013.
- [11] D. Phan, N. Pradhan, and M. Jankowiak. Composable effects for flexible and accelerated probabilistic programming in NumPyro. *arXiv:1912.11554*, 2019.
- [12] J. Bradbury, R. Frostig, P. Hawkins, et al. JAX: Composable transformations of Python+NumPy programs, 2018.
- [13] Y. LeCun. A path towards autonomous machine intelligence. OpenReview, 2022.
- [14] M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *CVPR*, pages 15619–15629, 2023.
- [15] A. Bardes, J. Ponce, and Y. LeCun. VICReg: Variance-invariance-covariance regularization for self-supervised learning. In *ICLR*, 2022.

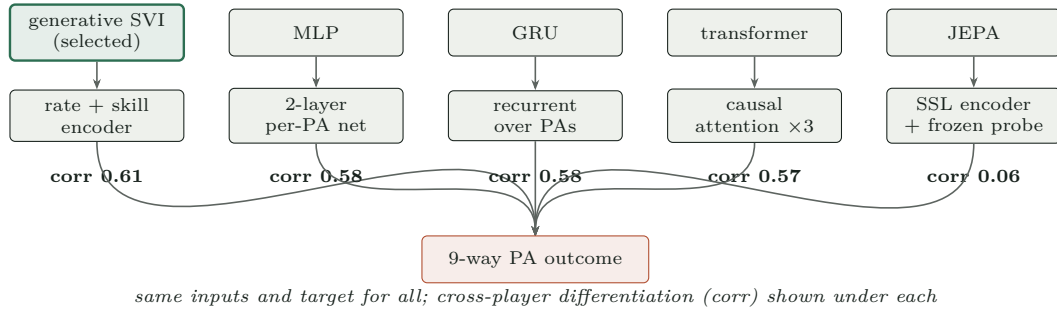


Figure 9. Architectures evaluated on identical inputs and target. The generative model with contact-quality features leads; discriminative nets cluster near 0.58; a frozen self-supervised probe collapses to 0.06 despite the best likelihood.